

Smart Parking System with Microservices and Embedded Integration

Davide Di Marco - davide.dimarco5@studio.unibo.it

Valerio Giannini - valerio.giannini@studio.unibo.it

June 12, 2025

Abstract

This project aims to design and implement an automated smart parking system based on a microservices architecture integrated with an embedded system. The goal is to manage real-time parking availability, recognize license plates using edge devices, and calculate fees upon exit. The system is designed to virtual payment methods and provides administrators with a real-time dashboard to monitor parking status.

Goals and Objectives

- Design and implement a modular architecture using microservices in Python using DDD.
- Integrate an embedded vision system (ESP32-CAM) for license plate recognition.
- Integrate NFC Embedded Module to accept payment
- Use a centralized database (MongoDB) to log parking sessions and payment histories.
- Develop RESTful APIs for each micro-services to allow communication between services and frontend.
- Design and develop an admin web dashboard for monitoring the system in python.

Work Plan

We will follow this work plan: In the initial phase, the team will analyze the requirements and apply Domain-Driven Design (DDD) to define the system domain. Then, we will proceed to implement and test the microservices and embedded components. Integration will be reached incrementally, with test-driven development adopted where possible. Finally, the system will be deployed using Docker containers.

Study and Design

- Analyze requirements and define system domain via DDD principles.
- Design microservices, database schema and system architecture (in an hybrid context: embedded plus microservices architecture)
- Design API specifications and inter-service communication model.

Development and Integration

- Implement microservices for the parking management, payment, and admin interface.
- Set up MongoDB and integrate with services.
- Configure and program ESP32-CAM for plate detection and NFC-Reader for payments.
- Develop the admin dashboard using the Flask Web framework.
- Implement REST APIs for all client-server interactions and eventually MQTT for the embedded part.

Testing and Deployment

Unit and integration tests will be written for each component to ensure correctness and system consistency. The entire system will be containerized using Docker for deployment and testing.